Shared Types and Data Reference

The `shared/` directory contains types and static data used by both the frontend app and potentially the portal.

`shared/types.ts` — Legacy Frontend Types

These are the types the **frontend** uses when working with route data. They are a simplified, legacy representation — the backend uses the richer `RouteDTO` type internally and converts to these before sending responses.

`Route`

```
interface Route {
  routeId: string;       // String version of the DB Route.id (e.g. "3")
  routeNodes: RouteNode[];
  factNodes: FactNode[];
}
```

A complete route with all its stops and facts loaded.

`RouteNode`

```
interface RouteNode {
  id: string;    // The node's key/coordinate (e.g. "1.0", "3.2") — NOT the DB
numeric id
  text: string;  // The interpretive text shown at this stop
}
```

Note: `RouteNode.id` here is the `key` field from the database (`RouteNode.key`), not the numeric primary key. This is a coordinate in the spritesheet system (see below).

`FactNode`

```
interface FactNode {
  id: string;          // The node key this fact belongs to (e.g. "4.2")
  text: string;        // The fact text shown after the route stop
  pollinator: string;  // The pollinator name (e.g. "Hummingbird", "Bee")
}
```

The `pollinator` field is what determines the pollinator revealed at the end of a route and what gets added to the player's collection.

Backend `RouteDTO` vs Frontend `Route`

The backend loads route data from the database as `RouteDTO` and converts it to the legacy `Route` shape before returning it to the frontend. The conversion happens in `mapRouteDTOToLegacyRoute()`

(`reshape.Utilites.ts`).

Backend `RouteDTO`	Frontend `Route` / `RouteNode` / `FactNode`
`id: number`	`routeId: string` (stringified)
`nodes[].key`	`routeNodes[].id`
`nodes[].text`	`routeNodes[].text`
`facts[].nodeId`	`factNodes[].id`
`facts[].text`	`factNodes[].text`
`facts[].pollinator`	`factNodes[].pollinator`

Spritesheet Coordinate System

Node keys use the format `"row.column"` to reference a position in the spritesheet image (`frontend/public/images/spritesheet_transparent.png`).

The special key `"0"` always refers to the starting node (position 0,0).

How the frontend uses coordinates:

In `PollinatorImage` component (`components.tsx`):

```
var split = coords.split(".");
var xCoord = parseInt(split[1]) * -100;   // column → CSS background-position-x
var yCoord = parseInt(split[0]) * -100;   // row    → CSS background-position-y
if (yCoord === 0) xCoord = 0;             // node "0" always maps to top-left
```

The spritesheet is a grid of 100×100px cells. The CSS `background-position` is set to show the correct cell via negative offset.

Example:

- Key `"4.2"` → row 4, column 2 → `backgroundPosition: "-200px -400px"` → shows the Hummingbird illustration

For the current pollinator grid positions, see ADDING_ROUTES.md or Contributor/UPDATING_POLLINATOR_IMAGES.md.

Where `shared/` is imported

Import location	What it uses
`frontend/src/app/services/routeService.ts`	`Route` type from `types.ts`
`frontend/src/app/home/page.tsx`	`Route` type from `types.ts`
`frontend/src/app/pollinator-collection/page.tsx`	`Route` type from `types.ts`

frontend/src/app/route/page.tsx	`Route` type from `types.ts`
backend/src/api/interfaces/api.interfaces.ts	`Route as LegacyRoute` type from `types.ts`
backend/src/api/Utilities/reshape.Utilites.ts	`Route as LegacyRoute` type from `types.ts` — converts `RouteDTO` to `Route`

The `shared/` directory is built with both the frontend and backend TypeScript compilations via `tsconfig.base.json` path aliases.

Rebuilding `types.js`

`shared/types.js` is the compiled JavaScript output of `types.ts`. It is committed to the repo and must be manually rebuilt whenever `types.ts` is changed.

```
# From the repo root
npx tsc --project shared/tsconfig.json
```

If you edit `types.ts` and forget to rebuild, the frontend and backend will compile against the updated TypeScript types but the runtime JavaScript will be stale — this causes silent mismatches at runtime.